

Relazione Progetto SO 2024/25

Ufficio delle Poste

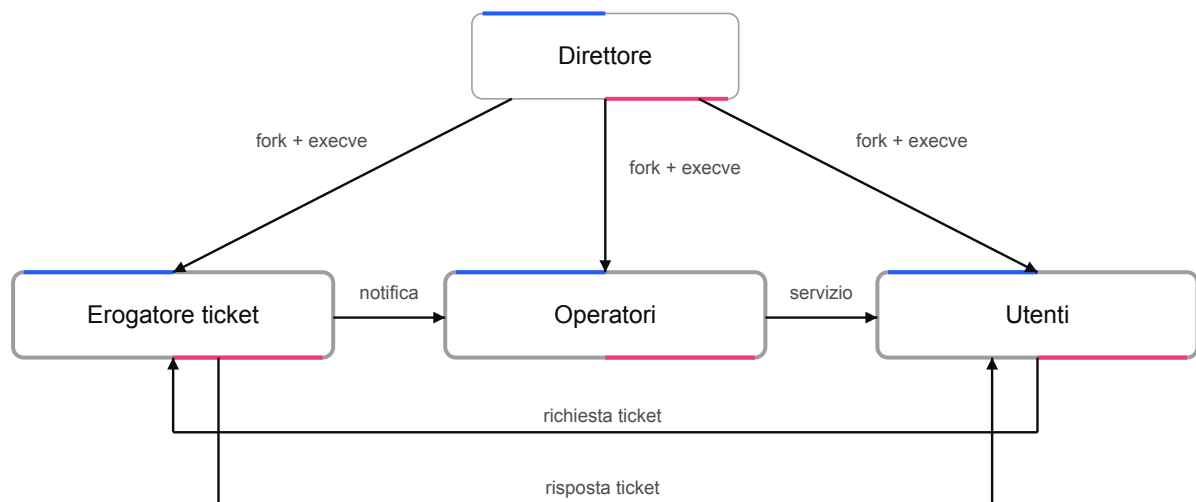
Rabottini Daniel, Riccardi Diego, Moramarco Giuseppe

Come eseguire il progetto

Per avviare la simulazione dalla directory principale del progetto:

- Compilare con **make**; il Makefile produce gli eseguibili in **bin/**.
- Avviare la configurazione standard con **make run**, oppure una prova mirata con **./bin/director_main config/config_timeout.conf**.
- Durante la simulazione richiedere nuovi utenti con **./bin/add_user_main N**: il direttore li crea all'inizio della giornata successiva.
- Modificare i parametri nei file **config/*.conf** senza ricompilare; le statistiche della run sono scritte in **data/stats.csv**.

1. Inizializzazione



add_user_main → coda aggiunta → Direttore · Direttore → data/stats.csv

- Il **direttore** legge e valida i parametri a runtime e crea memoria condivisa, semafori e code di messaggi System V.
 - Genera tramite **fork + execve** un erogatore ticket, gli sportelli condivisi, gli operatori specializzati e gli utenti iniziali.
 - Ogni figlio si collega alla memoria condivisa e segnala di essere pronto tramite **SEM_READY**; il direttore attende tutti prima della prima giornata.
-

2. Avvio della simulazione

- Il direttore coordina giornate di durata configurabile; un minuto simulato corrisponde a **N_NANO_SECS_PER_MIN** nanosecondi e la giornata contiene **MINS_PER_DAY** minuti.
- Operatori e utenti attendono semafori di inizio/fine giornata; non si usa attesa attiva.

1. Avvio e inizializzazione (Direttore)

- **director_main.c** invoca **director_init()**: **load_configuration()**, **create_shared_memory()**, **create_semaphores()**, **msgget()**, **stats_init()**, **window_init()**.
- **director_create_processes()**: **fork_ticket_dispenser()**, **fork_operators()**, **fork_users()**; il direttore aspetta **NOF_WORKERS + NOF_USERS + 1** segnali di pronto.
- Sono create sette code: richiesta e risposta ticket, notifica nuovo ticket, chiamata, completamento, disponibilità sportello e aggiunta utenti.

2. Inizio giornata (Direttore → Tutti)

- In **daily_routine()**, il direttore resetta i semafori giornalieri e le statistiche con **stats_reset_daily()**.
- Elabora le richieste ricevute sulla coda di **add_user_main** e attende che i nuovi processi si inizializzino.
- Assegna agli sportelli servizi a rotazione in base al giorno e sblocca **SEM_DAY_START_OP** e **SEM_DAY_START_US**.

3. Preparazione operatori e utenti

- **Operatori** (`operator_run()`): dopo il semaforo di avvio cercano uno sportello libero per il servizio di specializzazione; in assenza, aspettano notifiche sulla coda `notifWindowQueueId`. La specializzazione resta fissa.
- **Utenti** (`user_run()`): ciascuno ha una probabilità individuale `pServ` nell'intervallo configurato; quando partecipa genera casualmente da 1 a `MAX_REQUESTS` richieste e un orario di arrivo.

4. Richiesta ticket e servizio (per ogni richiesta utente)

a. Scelta del servizio

- Il processo utente verifica che il servizio sia assegnato ad almeno uno sportello, leggendo la memoria condivisa protetta da `SEM_WINDOWS`. Se non è offerto, registra la richiesta come non erogata.

b. Richiesta del ticket

- `user_get_ticket()`: `msgsnd` su `reqTicketQueueId`; il dispenser riceve con `msgrcv`.
- Il dispenser invia la risposta su `replyTicketQueueId`, indirizzata al PID dell'utente, e notifica il nuovo ticket su `newServiceId` con tipo legato al servizio.

c. Attesa della chiamata

- L'operatore del servizio preleva il ticket da `newServiceId` e invia la chiamata su `rcvServiceId`; l'utente attende il proprio identificativo di ticket.

d. Erogazione del servizio

- `simulate_service_time()`: durata casuale nell'intervallo $\pm 50\%$ della media del servizio richiesto.
- L'operatore invia il completamento su `doneServiceId`; l'utente aggiorna le statistiche tramite `user_update_stats()`, con `SEM_STATS`. La richiesta successiva parte solo dopo la precedente.

5. Pause e rilascio sportello (Operatori)

- Dopo un servizio l'operatore può interrompere il lavoro con probabilità `P_PAUSE`, entro `NOF_PAUSE` pause totali e solo se resta più di un operatore attivo per quel servizio.
- Aggiorna le statistiche protette da `SEM_STATS`, libera lo sportello con `release_window()` e notifica gli operatori in attesa tramite `notifWindowQueueId`.

6. Fine giornata

- Il direttore avanza `current_tick` per `MINS_PER_DAY` minuti; alla chiusura invia `SIGUSR1` agli utenti e agli operatori.
- Sblocca `SEM_DAY_END_US` e `SEM_DAY_END_OP`, poi aspetta `SEM_DAY_DONE` da tutti. I figli restano collegati alla memoria condivisa fino alla fine della simulazione.

7. Statistiche, terminazione e cleanup

- Il direttore stampa i dati giornalieri e, alla fine, quelli cumulativi. `stats_csv_begin()`, `stats_csv_append_daily()`, `stats_csv_append_final()` scrivono `data/stats.csv` con colonne Scope, Day, Service, Window, Metric, Value.
- Se gli utenti ancora in attesa a fine giornata superano `EXPLODE_THRESHOLD`, la causa è `explode`; altrimenti, dopo `SIM_DURATION` giornate, è `timeout`.
- `director_cleanup()` termina e attende i figli, quindi rimuove code, semafori e memoria condivisa.

3. Analisi dei test: configurazioni explode e timeout

Prove eseguite su Linux con i file in `config/`. Durata, soglia e utenti corrispondono a `SIM_DURATION`, `EXPLODE_THRESHOLD` e `NOF_USERS`. I conteggi sono esempi di singole esecuzioni e possono variare.

Configurazione	Durata	Soglia	Utenti	Servizi erogati	Non erogati	Attesa media	Terminazione
standard	5	11	300	289	75	43,16 min	explode, g. 1
timeout	2	1000	30	84	0	4,65 min	timeout, g. 2
explode	5	0	30	4	26	0,08 min	explode, g. 1

Test configurazione "timeout"

- **Obiettivo:** verificare la conclusione dopo `SIM_DURATION = 2` giorni, con `EXPLODE_THRESHOLD = 1000` a evitare una terminazione anticipata.
- **Risultato:** esito **timeout** al secondo giorno. La riga della tabella riproduce il CSV campione incluso in `data/stats.csv`: 44 utenti serviti, 84 servizi erogati, 0 non erogati, attesa media 4,65 minuti.

Test configurazione "explode"

- **Obiettivo:** forzare una coda residua con `EXPLODE_THRESHOLD = 0`, `NOF_WORKERS = 1` e `P_SERV_MIN = P_SERV_MAX = 100`.
- **Risultato:** nella prova Linux la soglia è stata superata al primo giorno e il programma ha riportato **explode**. I servizi non erogati comprendono richieste ancora in attesa e richieste non soddisfatte.

Configurazione standard

- Il file `configuration_file.conf` imposta 300 utenti, 8 operatori, 6 sportelli, 5 giorni massimi e soglia 11. Nella prova Linux la soglia è stata superata al primo giorno; pertanto non rappresenta un esempio di timeout.

Le statistiche sono inizializzate a ogni avvio: il CSV precedente viene sostituito. Per confrontare i risultati di più configurazioni occorre conservare separatamente il file prodotto da ciascuna esecuzione.